

MicroC/OS-II 용 디바이스 드라이버 작성①

문자 LCD 드라이버

총 3회의 연재를 통해 MicroC/OS-II라는 RTOS에서 동작할 수 있는 간단한 디바이스 드라이버를 소개하고자 한다. 이 글에서 소개하는 드라이버를 통해서 여러분은 실시간 커널용 디바이스 드라이버를 작성할 때 알아야 할 아주 중요한 다음 3가지 개념을 배울 수 있다. 이것들은 실시간 커널의 동작 자체를 이해할 때도 반드시 알아야 할 개념들이다.

글: 성원호/디오이즈 대표이사
whsung@dioiz.com

최근 업계와 학계에서 임베디드 시스템에 대한 관심이 확대되고 있는 가운데 임베디드 시스템을 동작시키는 소프트웨어의 핵심이라 할 수 있는 RTOS 역시 많은 관심을 끌고 있다. 현존하는 상용 또는 비상용 RTOS는 상당히 여러 가지가 있으나, 초보 임베디드 엔지니어는 물론이고 상당한 경험이 있는 엔지니어라도 처음으로 RTOS를 접해서 그 개념을 이해하고 적용해 보는

것은 쉬운 일이 아니다. 그 이유 중 첫째는 대부분의 RTOS가 32비트 시스템을 목표로 하고 있기 때문에 개인적인 관심만으로는 32비트 타깃 하드웨어와 그에 따른 개발환경을 갖추는 것이 비용상 많은 부담이 된다는 점이다. 둘째는 환경을 갖춘 뒤에도 실제 RTOS의 핵심을 효과적으로 이해할 수 있도록 도와주는 교재가 거의 없다는 점이다.

필자는 이 글을 통해 MicroC/OS-II라는 RTOS에서 동작할 수 있는 간단한 디바이스 드라이버를 소개하고자 한다. 이 연재에서는 소개할 드라이버는 인텔의 16비트 마이크로프로세서인 80c196kc가 장착된 실제 임베디드 시스템용이다. 이 연재에서 사용한 실시간 커널 실습키트는 (주)에이콘출판사(www.acornpub.co.kr)에서 판매하고 있는 DTB-196-BK라는 제품이다.

이 실습키트는 MicroC/OS-II와 이 연재에서 소개할 3가지 디바이스 드라이버를 간편하게 실습해 볼 수 있는 모든 소스코드와 컴파일 환경을 포함하고 있다. 즉, 이 실습키트와 PC만 있으면 즉시 RTOS의 세계에 입문할 수 있다.

MicroC/OS-II는 ROM화 가능하고, 용도에 따라 커널 크기를 다양하게 조절할 수 있으며, 실시간 응답을 보장하는 선점형 멀티태스킹 커널이다. 또 일관성 있게 코딩된 소스코드가 공개돼 있으므로 커널 내부와 동작원리를 쉽고 확실하게 이해할 수



MicroC/OS-II 실시간 커널 실습 키트

있다. MicroC/OS-II의 소스코드와 모든 설명은 참고서적[1]을 참조하기 바란다. 일단 MicroC/OS-II를 이용해서 실시간 커널의 구조를 이해하고 나면, 다른 상용 RTOS를 사용해서 응용 프로그램을 작성할 때도 많은 도움이 된다. 커널 내부 구조와 실시간 특성을 잘 이해하고 있을수록 더욱더 모듈화되고 효율적인 응용 프로그램을 작성할 수 있기 때문이다. 실제로 필자는 RTOS를 사용하고 있으면서도 RTOS의 강력한 기능을 충분히 사용하지 못하는 개발자를 많이 보았다. 그들은 단지 상용 RTOS가 제공해 주는 파일 시스템, GUI, TCP/IP 스택 등을 사용하기 위해서 RTOS를 사용하고 있을 뿐이었다. RTOS를 사용해서 얻을 수 있는 진정한 장점은 다음과 같다.

• 소프트웨어 개발과 유지보수성 향상

임베디드 시스템이 수행해야 할 각각의 기능을 담당하는 여러 개의 태스크가 멀티태스킹으로 동작하므로 소프트웨어 모듈화와 프로세서 사용 효율을 극대화할 수 있다. 또 각 태스크의 의존성이 크게 줄어들고 서로의 인터페이스만 정확히 정의하면 되므로 개발과 유지보수를 편리하게 해준다.

• 시스템 응답성 향상

RTOS 상에서는 태스크들이 철저하게 우선순위를 기반으로 동작하며, 각 태스크가 단순히 시간 지연을 위해 한 곳에 머물러 있을 때 다른 태스크가 프로세서를 사용할 수 있으므로 사용자 인터페이스 상호작용과 통신 흐름을 아주 원활하게 유지할 수 있다.

• 기존 응답성 유지

아주 강력한 장점이다. 이미 작성된 응용 프로그램에 새로운 태스크를 낮은 우선순위로 추가할 경우 시스템의 기존 응답성은 전혀 변동이 없다. 일반적인 순차 프로그래밍 방식에서는 꿈도 꿀 수 없는 장점이다. 이런 시스템에서는 함수 하나만 추가해도 전체 응답성을 다시 한 번 점검해봐야 한다.

MicroC/OS는 1992년에 처음 발표된 이후 수 백여 상용제품에 적용돼 그 안정성을 인정받고 있다. 국내에서도 MicroC/OS-II를 상용제품에 적용하면서 정식으로 라이선스를 받는 업체가 증가하고 있다. MicroC/OS-II는 32비트 시스템은 물론 16비트와 8비트 시스템에서도 동작할 수 있다는 큰 장점이 있다. 따

라서 대부분의 기존 RTOS가 32비트 시스템만을 지원하기 때문에 8비트와 16비트 시스템에서 RTOS를 사용할 엄두도 내지 못하는 엔지니어에게 아주 훌륭한 솔루션이 될 수 있다.

사실 8비트, 16비트 시스템에서도 펌웨어는 엄청나게 복잡해질 수 있으며, 이런 복잡한 코드로 인해 밤잠을 설쳐가며 코드를 수정하느라 여념이 없는 펌웨어 엔지니어를 흔히 찾아볼 수 있다. 이들에게 MicroC/OS-II를 적용해볼 것을 적극 권장한다. RTOS를 사용해서 멀티태스킹 프로그램을 작성하면 복잡하게 얽혀있는 펌웨어 소프트웨어 구조를 획기적으로 간결하게 만들 수 있다. MicroC/OS-II는 수많은 8비트, 16비트 프로세서에 잘 동작한다.

MicroC/OS-II는 그동안 OS 주변 소프트웨어인 파일시스템, 그래픽 인터페이스, 네트워크 스택 등을 지원하지 않는다는 단점 때문에 지적을 받아왔다. 하지만 최근 들어 uC/FS, uC/GUI, uC/FlashLoader 등 임베디드 시스템에 최적화된 소프트웨어 제품들을 지속적으로 출시하고 있고, 이 기사가 연재되는 9월이면 uC/TCP-IP라는 네트워크 제품이 출시될 예정이다. 이들 제품은 Embedded Linux나 Windows CE 등 덩치 큰 OS를 적용하기 힘든 로우엔드 임베디드 시장을 목표로 설계했기 때문에 CPU 성능이 낮고 메모리 제약이 심각한 제품에도 적용할 수 있는 진정한 의미의 임베디드 소프트웨어다.

이 연재에서는 MicroC/OS-II는 물론이고 다른 실시간 커널에서도 사용할 수 있는 간단한 디바이스 드라이버 3가지를 소개한다. 이들 드라이버는 “Embedded Systems Building Blocks, 2nd Edition”이라는 책에 소개된 것들 중 일부다. 이 책은 임베디드 시스템에서 아주 유용하게 사용할 수 있는 9가지 빌딩 블록 즉, 소프트웨어 모듈을 설명한다. 하지만 이 책에 소개된 코드는 PC를 기준으로 작성됐기 때문에 실제 임베디드 시스템에서 사용하기 위해서는 약간의 포팅이 필요하다.

이 글에서 소개하는 드라이버를 통해서 여러 분은 실시간 커널용 디바이스 드라이버를 작성할 때 알아야 할 아주 중요한 다음 3가지 개념을 배울 수 있다. 이것들은 실시간 커널의 동작 자체를 이해할 때도 반드시 알아야 할 개념들이다.

1. 태스크 사이의 공유자원 처리

실시간 커널 상에서 동작 중인 모든 태스크들은 우선순위가 더 높은 태스크에 의해 언제든지 선점될 수 있다. 이 때 선점된 태스크가 공유자원을 사용 중이었다면 문제가 발생할 수 있다.

디바이스 드라이버는 여러 태스크가 사용할 수 있는 하드웨어 공유자원을 처리하는 소프트웨어이기 때문에 이런 문제는 얼마든지 발생할 수 있다. 실시간 커널이 제공하는 세마포어 서비스를 사용하면 이런 공유자원 문제를 해결할 수 있다. 공유자원을 보호하는데 사용하는 특수한 형태의 세마포어를 뮤텍스(Mutex)라고 부르기도 한다.

2. 디바이스 드라이버 전용 태스크

경우에 따라서 디바이스를 모니터링하는 전용 태스크를 뒤야 할 경우가 있다. 이 개념은 디바이스가 하드웨어적으로 인터럽트를 지원하지 않을 경우나 인터럽트를 지원하지만 폴링(polling) 처리가 더 효과적일 경우에 사용하는 기법이다. 이번 연재에서 예를 들 매트릭스 키패드의 인터페이스는 하드웨어적으로 인터럽트가 발생하지 않는다. 주기적으로 실행하면서 키패드를 모니터링하는 전용 태스크를 뒤서 아주 적은 CPU 오버헤드만으로 동작하는 키패드 드라이버를 구현한다.

3. 인터럽트 서비스 루틴에서 태스크로 신호 보내기

인터럽트 서비스 루틴에서 인터럽트가 발생한 사실을 알리기 위해 태스크로 신호를 보내는 것은 실시간 커널용 디바이스 드라이버 작성시 가장 중요한 개념이다. 시리얼 포트 드라이버를 통해 이 개념을 설명한다.

문자 LCD 드라이버

첫 연재에서는 문자 LCD(Liquid Crystal Display)를 구동할 수 있는 소프트웨어 모듈을 설명한다. 이 드라이버는 Hitachi HD44780 도트 매트릭스 LCD 컨트롤러와 호환하는 모든 문자 LCD 모듈에 사용할 수 있다. 이 드라이버는 다음과 같은 기능을 제공한다.

- 80문자까지 표시 가능한 LCD모듈 제어
- ASCⅡ 문자 표시
- ASCⅡ 문자열 표시
- 5×7 도트 매트릭스 기호를 8개까지 정의
- 바그래프 표시

문자 LCD 모듈은 LCD와 구동회로로 구성된다. 문자 표시장

치는 16~40개의 문자로 구성된 라인을 포함하며, 보통 1~4개의 라인이 있다. 각 문자 블록은 5×8의 도트 매트릭스로 구성되고, ASCⅡ 문자와 제한된 종류의 기호를 표시할 수 있다. 여기서 문자 표시 모듈의 소프트웨어 인터페이스 모듈을 제공할 것이다. 문자 모듈은 다음과 같은 임베디드 시스템에서 널리 사용되고 있다.

- 에어컨
- 음향앰프
- 팩스기기
- 레이저 프린터
- 의료기기
- 보안 시스템
- 전화기

멀티태스킹 RTOS 환경이 아닌 일반적인 전경/배경(Foreground/Background) 시스템의 경우는 LCD에 접근하는 함수를 작성할 때, 이들 함수가 중첩되어 여러 번 호출될 경우를 염려할 필요가 없다. 여기서 전경이란 인터럽트 발생시 인터럽트 레벨에서 실행하는 코드를 말하고, 배경은 인터럽트 상태가 아닌 평상시 코드 실행상태를 말한다. 한 마디로 전경/배경 시스템이란 RTOS를 사용하지 않는 일반적인 순차실행 방식의 프로그램을 사용하는 시스템을 말한다. 따라서 전경 즉, 인터럽트 서비스 루틴 안에서 LCD 접근 함수를 호출하지 않는 한, 배경에서는 LCD 접근 함수를 호출해서 아무 문제없이 LCD를 사용할 수 있다.

하지만 RTOS를 사용하는 멀티태스킹 환경에서는 얘기가 달라진다. 멀티태스킹 환경에서는 통상 태스크(Task)라고 부르는 배경이 여러 개 동작한다. 한 태스크가 동작하고 있을 때 우선순위가 더 높은 태스크가 현재 실행 중인 태스크를 선점해서 동작할 수 있다. 이때 기존의 태스크가 LCD 같은 공유자원에 접근해서 그 자원을 사용 중이고, 새로 실행하게 된 태스크도 공유롭게 같은 공유자원에 접근하고자 하면 문제가 발생한다. 즉, LCD에 깨진 글자가 보이는 현상이 생길 수 있다는 말이다.

멀티태스킹 환경에서는 이런 공유자원 충돌 문제가 발생하지 않도록 주의를 해야 한다. 모든 일에는 장단점이 있듯이 RTOS를 사용하면 장점도 있지만 공유자원 같은 문제를 고려해야 한다는 단점도 있다. 하지만 단점에 비해 장점이 더 크기 때문에 RTOS를

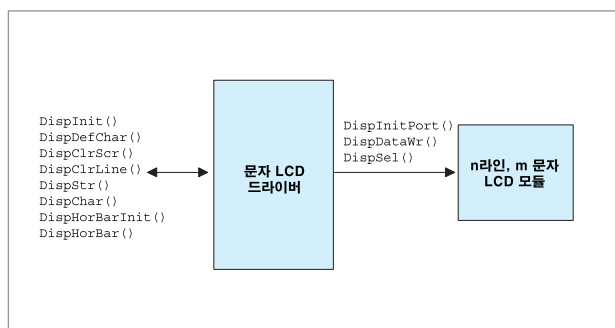


그림 1. 문자 LCD 드라이버가 제공하는 함수들

사용하는 것이다. 공유자원 문제를 해결할 수 있는 가장 바람직한 방법은 공유자원 충돌 문제를 원천적으로 해결한 함수를 만들어서 각 태스크가 사용하도록 하는 것이다. 이때 사용하는 것이 세마포어라는 커널 오브젝트다. 세마포어에 대한 개념과 구현 원리는 기존 서적[1]을 참조하기 바란다. 어쨌든 문자 LCD는 공유자원의 대표적인 예이며 이번 연재에서 다루고자 하는 내용이다.

문자 LCD 드라이버는 멀티태스킹 환경에서 각 태스크가 사용할 수 있는 여러 가지 함수를 제공한다. 이들 함수 안에 세마포어를 캡슐화해서 각 태스크들은 자원 충돌 문제를 신경 쓰지 않고 자원에 접근할 수 있다. 이렇게 자원 충돌 문제없이 사용할 수 있는 함수를 재진입 가능 함수(Reentrant function)라고 한다. 재진입 가능 함수는 LCD나 시리얼 포트 같은 하드웨어 공유자원뿐 아니라 메모리 상에 존재하면서 공유되는 변수, 행렬, 구조체 등도 안전하게 사용할 수 있도록 해준다.

문자 LCD 드라이버는 그림 1에 보인 여러 가지 함수를 제공한다. 그림 1에서 왼쪽에 있는 함수들이 태스크 레벨에서 사용할 수 있는 함수다. 응용 프로그램에서는 이들 함수를 이용해서 문자 LCD 기반의 사용자 인터페이스를 구현할 수 있다. 가운데 있는 3가지 함수는 문자 LCD가 실제 어떻게 하드웨어적으로 인터페이스되어 있는지에 따라 그 내용이 달라지는 하드웨어 의존적인 부분이다. 원래 드라이버는 PC의 ISA 버스에 장착된 범용 8255 인터페이스 카드에 문자 LCD를 단 것으로 가정하고 있다. 하지만 실습 키트에서는 문자 LCD가 Memory mapped 방식으로 인터페이스되어 있기 때문에 수정이 필요하다.

문자 LCD 드라이버 소스코드에 포함된 각 함수를 3가지 부류로 나누어 설명할 것이다. 이 연재는 LCD 모듈을 구체적으로 어떻게 사용하는지에 대한 설명은 생략한다. 그런 설명이 필요한 독자는 이미 수없이 출간된 기존 마이크로프로세서 입문서를

참조하기 바란다. 여기서는 멀티태스킹 RTOS 상에서 공유자원 충돌 문제를 방지할 수 있는 디바이스 드라이버 작성법 설명에 초점을 맞춘다. 그리고 지면 관계상 드라이버 소스코드를 전부 게재할 수 없으므로 세마포어를 사용해서 공유자원을 보호하는 개념과 직접적인 관계가 있는 부분의 코드만 게재하겠다.

문자 LCD 초기화 함수

우선 LCD 드라이버를 사용하기 위해 가장 먼저 호출해야 하는 초기화 함수다.

```

void DispInit (INT8U maxrows, INT8U maxcols)
{
    DispInitPort();
    DispMaxRows = maxrows;
    DispMaxCols = maxcols;
    DispSem = OSSemCreate(1);

    DispSel(DISPLAY_SEL_CMD_REG);
    OSTimeDlyHMSM(0, 0, 0, 50);
    DispDataWr(DISPLAY_CMD_FNCT);
    OSTimeDly(2);
    DispDataWr(DISPLAY_CMD_FNCT);
    OSTimeDly(2);
    DispDataWr(DISPLAY_CMD_FNCT);
    OSTimeDly(2);
    DispDataWr(DISPLAY_CMD_FNCT);
    OSTimeDly(2);

    DispDataWr(DISPLAY_CMD_ON_OFF);
    DispDataWr(DISPLAY_CMD_MODE);
    DispDataWr(DISPLAY_CMD_CLS);
    OSTimeDly(2);
}
  
```

이 함수에서는 LCD 모듈을 초기화할 때 필요한 시간지연을 위해 MicroC/OS-II가 제공하는 OSTimeDly() 함수를 호출한다. 따라서 먼저 MicroC/OS-II를 기동한 뒤에 이 함수를 호출해야 한다. 또한 LCD에 배타적인 접근을 보장하기 위해 DispSem이라는 세마포어를 생성한다. 실제 세마포어를 생성하는 코드는 다음과 같다.


```
DispSem = OSSemCreate(1);
```

여기서 세마포어의 초기값을 1로 설정하는 점에 주목하기 바란다.

DispSem은 생성된 세마포어의 메모리 위치를 가리키는 포인터이며, 드라이버 소스코드의 선두 부분에 다음과 같이 선언되어 있다.

```
static OS_EVENT *DispSem; /* LCD용 세마포어 */
```

세마포어는 궁극적으로 C 구조체에 불과하며 이 구조체 안에는 카운터와 태스크 대기 리스트가 있다. 이 카운터에 값이 있을 때 세마포어를 획득하고자 하는 태스크는 세마포어를 얻을 수 있다. 태스크가 세마포어를 소유할 때 카운트 값은 1 줄어든다. 카운터 값이 0일 때는 태스크가 세마포어를 획득할 수 없다. 문자 LCD는 공유자원이고 한 번에 한 태스크만 접근해야 한다. 따라서 세마포어 생성시 초기값을 1로 설정하면 최초로 LCD에 접근한 태스크만 세마포어를 획득해서 공유자원을 사용할 수 있다. 공유자원을 다 사용하고 세마포어를 돌려줄 때 다른 태스크가 그 세마포어를 받아서 공유자원을 사용할 수 있다. 만약 다른 RTOS 상에서 이 드라이버를 사용하고자 한다면 세마포어 관련 함수와 시간지연 함수를 적절히 바꿔줘야 한다.

태스크가 호출하는 함수

초기화 함수를 제외하고 태스크가 호출할 수 있는 각 함수들을 들여다보면, 모두 다음과 같이 자원에 접근하기 전에 세마포어를 획득하고 자원을 다 사용한 뒤 세마포어를 반환하는 구조로 되어있다.

```
void Disp??? (void)
{
    INT8U err;

    OSSemPend(DispSem, 0, &err);
    ...
    ...
    ...
    OSSemPost(DispSem);
}
```

구체적으로, 임의의 좌표에 문자열을 표시하는 함수는 다음과 같다.

```
void DispStr (INT8U row, INT8U col, char *s)
{
    INT8U i;
    INT8U err;

    if (row < DispMaxRows && col < DispMaxCols) {
        OSSemPend(DispSem, 0, &err);
        /* LCD용 세마포어를 획득한다 */
        DispCursorSet(row, col);
        /* 커서를 ROW/COL에 위치시킨다 */
        DispSel(DISPLAY_SEL_DATA_REG);
        i = col; /* 최대 허용 가능한 값으로 카운터를 설정한다 */
        while (i < DispMaxCols && *s) {
            /* (str+limit)와 DispMaxCols 사이에 문자를 쓴다 */
            DispDataWr(*s++); /* 표시장치에 문자를 쓴다 */
            i++; /* 카운터 증가 */
        }
        OSSemPost(DispSem); /* 세마포어를 반납한다 */
    }
}
```

위 함수에서 OSSemPend()와 OSSemPost()를 제외한 다른 함수들은 LCD 하드웨어에 직접 접근하는 함수다. 따라서 문자열을 표시하고자 하는 태스크가 DispStr() 함수를 호출해서 하드웨어에 접근하고 있는 동안 다른 태스크가 이 태스크를 선점해서 다시 LCD에 접근할 때 문제가 생길 수 있고, 세마포어를 이용해서 이런 문제를 방지하는 것이다.

태스크가 호출하는 LCD 접근 함수는 모두 이런 구조이기 때문에 세마포어를 통해 LCD에 접근할 때 발생할 수 있는 공유자원 충돌 문제를 막을 수 있다. 그 원리를 순서대로 설명한다.

1. 태스크가 문자 LCD에 접근하고자 할 때 LCD 드라이버가 제공한 함수를 호출한다.

2. 함수 안에서 문자 LCD에 하드웨어적으로 접근하기 전에 OSSemPend()라는 함수를 호출해서 DispSem이라는 세마포어를 얻는다. 태스크가 최초로 LCD에 접근할 경우 OSSemPend() 함수는 세마포어를 얻은 채 즉시 리턴한다. 이때 커널은 다른 태

스크가 DispSem을 사용할 수 없도록 만들어 놓는다. 즉, DispSem의 소유권이 최초로 LCD에 접근한 태스크로 넘어간 것이다.

3. 모든 태스크는 문자 LCD에 접근할 때 반드시 DispSem을 얻어야 하므로 먼저 DispSem을 얻은 태스크는 마음 놓고 LCD에 접근해서 데이터를 쓰고 읽을 수 있다. DispSem을 소유한 태스크가 LCD와 상호작용하고 있을 때 우선순위가 더 높은 다른 태스크가 깨어나서 DispSem을 소유한 태스크를 선점하고, LCD에 접근하기 위해 DispSem을 얻고자 OSSemPend()를 호출하면, 커널은 다른 태스크가 이미 DispSem을 소유하고 있다는 사실을 알아내고 이 태스크를 세마포어의 대기 리스트에 등록한다. 두 번째 태스크는 대기상태로 들어가고 원래 실행하고 있던 태스크로 다시 문맥전환이 일어난다.

4. 처음 태스크는 LCD와 남은 상호작용을 마무리한다. 남은 마무리 기간 동안 또 다른 태스크가 깨어나서 2, 3번 과정을 반복할 수도 있다.

5. 최초 태스크가 LCD와의 상호작용을 끝내고 OSSemPost()를 호출하는 순간, 커널은 DispSem의 대기 리스트에 지금 막 LCD 사용을 끝낸 태스크 보다 우선순위가 높은 태스크가 대기 중인 사실을 알아낸다.

6. 커널은 대기 중이던 태스크를 세마포어 대기 리스트에서 제거하고 준비상태로 만든 뒤 바로 전에 실행을 멈췄던 곳부터 재실행시킨다. 즉, 태스크가 세마포어를 얻으려다 멈춘 곳부터 다시 실행하고 이제는 세마포어를 획득한 상태가 되어 LCD에 접근할 수 있다.

위의 과정을 우리가 쉽게 생각할 수 있는 예를 들어 다시 설명한다. 공유자원은 밀폐된 방에 들어 있고 그 방문은 자물쇠가 채워져 있다. 그 공유자원이 구체적으로 무엇인지는 알 필요가 없다. LCD도 될 수 있고, 시리얼 포트 또는 병렬포트도 될 수 있다. 공유자원을 사용하고자 하는 태스크는 그 방문을 열 수 있는 열쇠가 필요하다. 열쇠는 1개만 있으며(세마포어 카운트 값을 1로 초기화하는 이유가 여기에 있다), 처음에는 방문 앞에 놓여있다. 이 열쇠를 세마포어로 간주할 수 있다. 태스크가 공유자원을 사용하기 위해 방문 앞에 놓인 열쇠를 이용해서 방문을 열고 열쇠를 든 채로 방으로 들어간다. 방으로 들어간 태스크는 문을 잠근다. 그 안에서 마음 놓고 공유자원을 사용한다. 열쇠 즉, 세마포어를 소유한 채로 공유자원을 혼자서 사용할 수 있게 된 것이

다. 다른 태스크가 공유자원을 사용하기 위해 방문 앞에 도착하지만 문을 열 수 있는 열쇠가 없기 때문에 문 앞에서 기다린다. 다른 태스크들이 방문 앞에 도착해도 마찬가지다. 공유자원을 다 쓴 태스크는 방에서 나와 문을 잠그고, 방문 앞에서 대기 중인 태스크들이 있을 경우는 이들 태스크 중에서 가장 우선순위가 높은 태스크 또는 가장 먼저 기다리기 시작한 태스크한테 열쇠를 넘겨준다.

하지만 대부분의 경우는 우선순위가 높은 태스크한테 주기 마련이다. 대기 중인 태스크가 없을 경우는 열쇠를 문 앞에 놓고 다른 일을 하기 위해 자리를 뜬다. 이때 공유자원을 다 사용한 태스크가 깜박하고 열쇠를 가진 채 다른 곳으로 가버리면 아무 태스크도 그 공유자원을 사용할 수 없게 된다. 그 공유자원이 LCD인 경우는 LCD 상에 문자가 안보이게 될 것이다. 이점을 특히 주의해야 한다. 지금 예에서 살펴봤듯이 공유자원 보호를 위한 세마포어를 개념상 열쇠로 생각할 수 있기 때문에 흔히 열쇠기호로 표시한다.

이런 방식을 통해 여러 개의 태스크가 하나뿐인 LCD 자원을 데이터 충돌 없이 사용할 수 있는 것이다. 우선순위가 낮은 태스크라도 공유자원에 먼저 접근했다면 그 자원을 다 사용하고 세마포어를 반납할 때까지 다른 태스크가 방해하지 않도록 커널이 보호해 준다. 이때 우선순위 전도 문제(priority inversion)가 발생할 수 있는데, 이런 문제를 해결하기 위한 다른 형태의 커널 오브젝트를 뮤텍스(Mutex)라고 한다. 하지만 여기서는 설명을 생략한다. 우선순위 전도 문제와 뮤텍스에 대한 설명은 참조서적[1]을 참조하기 바란다.

하드웨어 인터페이스에 의존적인 함수

다음은 하드웨어 의존적인 3가지 함수에 대한 설명이다.

아래 함수는 LCD 모듈에 8비트 데이터를 쓰기 위한 함수다. 이 함수로 데이터를 쓰기 위해서는 쓰고자 하는 번지를 먼저 dataPtr에 설정해야 한다.

```
void DispDataWr (INT8U data)
{
    INT8U dly;

    *dataPtr = data;
    for (dly = DISP_DLY_CNTS; dly > 0; dly--) {
        /* 최소한 40 usec 지연 */
    }
}
```

```

        DispDummy();
    }
}

```

dataPtr에 번지를 설정하는 함수는 다음과 같다.

```

void DispSel (INT8U sel)
{
    if (sel == DISP_SEL_CMD_REG) {
        dataPtr = (BYTE *)LCD_CW;
    } else {
        dataPtr = (BYTE *)LCD_DW;
    }
}

```

여기서 LCD_CW와 LCD_DW는 각각 0x0300와 0x0301이다. 즉, 이 함수는 명령어를 쓸지 데이터를 쓸지 결정하는 역할을 한다. 나머지 함수는 8255 포트를 초기화하는 함수인데 이 실습키트에서는 8255 포트를 사용하지 않기 때문에 다음과 같이 빈 채로 됐다.

```

void DispInitPort (void)
{
    /* DTB-196은 8255 포트를 초기화할 필요가 없으므로 이 부분은 아무 코드로 삽입하지 않는다 */
}

```

문자 LCD 드라이버를 사용하는 예제

다음 사진은 실습키트에서 MicroC/OS-II와 함께 예제프로그램을 실행한 예를 보여준다. 후반부에 예제 프로그램 소스코드를 게재했으니 참조하기 바란다.

첫 라인에는 임의의 위치에 각자 맡은 문자를 표시하는 태스크가 9개가 동작하고 있다. 아래 라인 좌측에는 현재 CPU 사용율과 초당 문맥전환 회수를 표시하는 태스크가 동작하고 있다. 아래 라인 우측에는 문자 LCD 드라이버가 제공하는 바그래프 기능을 이용해서 동적으로 바그래프를 그리는 태스크가 동작하고 있다. 총 12개의 사용자 태스크가 아주 빠른 속도로 동작하고 있으며, 현재 CPU 점유율은 39%, 문맥전환은 초당 681번 발생하고 있다는 것을 보여준다. 따라서 아직 60% 정도의 CPU 여



MicroC/OS-II와 함께 예제프로그램을 실행한 예

유가 있다는 사실을 알 수 있다. 실제 멀티태스킹 동영상은 <http://www.dioiz.com/download/dtb-196.WMV>을 참조하기 바란다. 여기서 동작하는 모든 태스크들은 문자 LCD 드라이버에서 제공하는 함수를 호출하기 때문에 LCD상의 문자가 깨지는 일 없이 각 태스크가 원하는 위치에 문자를 표시할 수 있다. 예제 프로그램 소스코드를 보면 각각의 태스크들은 다른 태스크에 개의치 않고 LCD 관련 함수를 호출하고 있다. 따라서 전적으로 드라이버 레벨에서 LCD 자원의 배타적 접근을 관리하고 있다는 것을 알 수 있다.

이상으로 문자 LCD 드라이버를 통해 RTOS 상에서 각 태스크들이 공유하는 자원을 어떻게 보호해야 할 지 알아보았다. 이 개념은 굳이 하드웨어 공유자원에 국한해서 사용하는 것이 아니다. 앞서도 언급했듯이 메모리 상에 존재하는 여러 공유 변수나 구조체, 데이터 버퍼에도 적용할 수 있다. 물론 빠른 시간 안에 메모리를 참조해서 결과를 계산할 경우는 간단히 인터럽트를 금지하고 해제하는 방법을 사용하면 된다. 하지만 메모리 상에 존재하는 대용량의 데이터에 접근해서 많을 계산을 수행할 경우는 인터럽트를 막는 것이 시스템 응답성에 영향을 줄 수 있으므로, 그럴 경우는 세마포어를 사용하는 것이 바람직하다. 세마포어는 공유자원의 배타적 접근을 제어하는 용도 외에도 몇 가지 용도가 더 있으며 RTOS 상에서 아주 중요한 역할을 하는 커널 오브젝트다. 후에 있을 연재에서 세마포어를 이용한 드라이버의 예가 더 있으므로 그때 가서 더 자세한 설명을 하겠다. ^{RTtime}

저작권

이 글에 게재한 일부 내용과 소스코드는 한국내 판권자인 (주)에이콘출판사의 허가를 받아 다음 서적에서 인용했습니다.

- [1] "MicroC/OS-II 실시간 커널, 제2판" (ISBN 898997533-6)
- [2] "Embedded Systems Building Blocks, 2nd Edition" (ISBN 89-951540-4-7)

[예제 프로그램]

```

#include "..\\includes.h"
#include "..\\drivers\\lcd.h"
#include "..\\drivers\\CommRtos.h"

/*****
/*          상수 선언          */
*****/

#define TASK_STK_SIZE 100
/* 각 태스크의 스택 크기 (16비트 스택) */
#define N_TASKS 9 /* 같은 코드의 태스크가 수행되는 수 */

#define KEY_ESC 0x1b

#define CHAR_TASK_DELAY 1
#define BAR_TASK_DELAY1 1
#define BAR_TASK_DELAY2 1

/*****
/*          전역 변수 선언          */
*****/
OS_STK TaskStartStk[TASK_STK_SIZE];
/* TaskStart()함수용 스택영역 */
OS_STK barTaskStk[TASK_STK_SIZE];
/* barTask()함수용 스택영역 */
OS_STK commTaskStk[TASK_STK_SIZE];
/* barTask()함수용 스택영역 */
OS_STK TaskStk[N_TASKS][TASK_STK_SIZE];
/* charTask()함수용 스택영역 */
BYTE TaskData[N_TASKS];
/* charTask()로 전달되는 파라미터용 배열 */

/*****
/*          함수 원형          */
*****/
void TaskStart(void *data);
void charTask(void *data);
void barTask(void *data);
void commTask(void *data);

/*****
/*          외부 함수 원형          */
*****/

```

```

extern void TimeTickInit(void);

void main (void)
{
    OSInit(); /* uC/OS-II를 초기화한다 */
    /* 최초 태스크 생성 */
    OSTaskCreate(TaskStart, (void *)0, (void *)&TaskStartStk
[TASK_STK_SIZE - 1], 0);
    OSStart(); /* uC/OS-II 기동 */
}

void TaskStart(void *pdat)
{
    BYTE i, ret;
    BYTE sData;
    char * sPtr;
    char dispBuf[30];

    disable(); /* 인터럽트 비활성화 */
    TimeTickInit(); /* 타임 틱용 타이머 초기화 */
    enable(); /* 인터럽트 활성화 */

    CommInit();
    DispInit(2,16); /* LCD 드라이버를 2*16 용으로 초기화한다 */
    DispHorBarInit(); /* LCD의 CG램을 프로그램한다 */
    DispStr(0,0,"System init..");
    DispStr(1,0,"Please wait..");

    OSStatInit();
    DispClrScr();

    ret = 0;

    for(i=0;i<N_TASKS;i++) /* N_TASKS만큼 동일 태스크를 생성한다 */
    {
        TaskData[i] = i;
        /* 각 태스크가 처음으로 표시할 데이터를 넘겨준다 */
        ret += OSTaskCreate(charTask, (void *)&TaskData[i],
(void *)&TaskStk[i][TASK_STK_SIZE - 1], i+1);
    }
    ret += OSTaskCreate(barTask, (void *)0, (void
*)&barTaskStk[TASK_STK_SIZE - 1], i+2);
    ret += OSTaskCreate(commTask, (void *)0, (void

```



```

*)&commTaskStk[TASK_STK_SIZE - 1], i+3);

if(ret)
{
    /* ret 변수에 어떤 값이 있으면 태스크 생성도중 에러가 발생한 것임 */
    DispStr(1, 0, (char *) "Prio Err");
    OSTimeDlyHMSM(0, 0, 5, 0);
}

i = 0;
/* 태스크생성 임무를 완료했으므로 무한루프를 반복한다 */
while(1)
{
    /* 1초 간격으로 CPU 사용율과 문맥전환 회수를 표시 */
    OSTimeDlyHMSM(0, 0, 1, 0); /* 1초 지연 */
    sprintf(dispBuf, "%02d%%,%04d", OSCPUUsage,
    OSCtxSwCtr);
    DispStr(1, 0, dispBuf);
    OS_ENTER_CRITICAL();
    OSCtxSwCtr = 0;
    OS_EXIT_CRITICAL();
    sprintf(dispBuf, "This is first example : %03d\r",
    i++);
    sPtr = dispBuf;
    while(*sPtr)
    {
        ret = CommPutChar(*sPtr++, 0);
    }
}

void charTask(void *pdat)
{
    BYTE val, cnt;

    val = *(BYTE *)pdat;
    val %= 10;
    val += '0';

    for (;;)
    {
        /* timer1으로 부터 글자를 표시할 랜덤한 위치를 읽어온다 */
        cnt = (BYTE)(timer1%16);

        /* 타이머에서 읽은 값을 LCD좌표로 변환해서 1문자를 디스플레이한다 */
        DispChar(0, cnt, val);

        /* 1 Time tick 동안 대기상태로 들어간다 */
        /* 이때 다른 낮은 순위의 태스크가 실행에 들어간다 */
        OSTimeDly(CHAR_TASK_DELAY);
    }
}

```

```

}
}

void barTask(void *data)
{
    BYTE i;
    char str[5];

    data = data;

    i = 0;

    for (;;) {
        for(i=0; i<=30; i++)
        {
            /* 0부터 30까지의 값을 이용해서 바그래프를 출력한다 */
            DispHorBar(1, 8, i);
            sprintf(str, "%02d", i);
            DispStr(1, 14, str);
            OSTimeDly(BAR_TASK_DELAY1);
        }
        OSTimeDly(BAR_TASK_DELAY2);
        for(i=29; i>0; i--)
        {
            /* 29부터 0까지의 값을 이용해서 바그래프를 출력한다 */
            DispHorBar(1, 8, i);
            sprintf(str, "%02d", i);
            DispStr(1, 14, str);
            OSTimeDly(BAR_TASK_DELAY1);
        }
        OSTimeDly(BAR_TASK_DELAY2);
    }
}

void commTask(void *data)
{
    INT16U cnt;
    INT8U sData, err;
    char dispBuf[30];
    INT8U flag = 1;

    cnt = 0;
    for(;;){
        sData = CommGetChar(0, &err);
        if(sData==KEY_ESC) asm rst;
        sprintf(dispBuf, "%05u", cnt++);
        DispStr(1, 3, dispBuf);
    }
}

```